

sanitizes injected tokens while maintaining the utility of the sanitized context for the target task. We note that PISanitizer does not fully restore the utility of contaminated contexts to the no-attack level for certain tasks such as code generation. We suspect the reason is that PISanitizer cannot perfectly remove the injected prompt, i.e., a small number of clean tokens are also occasionally removed. These tasks can be relatively more sensitive to such removals.

PISanitizer accurately sanitizes injected tokens: Table 14 (in Appendix) shows the precision (fraction of sanitized tokens that are injected), recall (fraction of injected tokens that are sanitized), and F1-score (harmonic mean of precision and recall) of PISanitizer when sanitizing injected prompts. These metrics are computed at the token level and reflect how accurately PISanitizer identifies and removes injected tokens. When there are no attacks, we find that PISanitizer removes 0.81 tokens on average (over clean contexts on six datasets), thereby maintaining target task utility. Under prompt injection attacks, on average, PISanitizer achieves 0.80 precision, 0.90 recall, and 0.82 F1-score. In general, PISanitizer can achieve a relatively high precision and recall for sanitizing injected tokens in a context with thousands of tokens. Our experimental results demonstrate that (1) only a small number of clean tokens are mistakenly sanitized, allowing PISanitizer to maintain target task utility, and (2) most of the injected tokens are accurately sanitized, making the remaining injected tokens ineffective, resulting in near-zero ASR as shown in Table 1.

PISanitizer generalizes across different LLMs: As shown in Table 2, PISanitizer is effective across backend LLMs with different architectures, parameter sizes, and accessibility (open-source and closed-source). Note that we consistently use Llama-3.1-8B-Instruct to perform context sanitization. The results show that PISanitizer consistently reduces ASR to nearly zero. We observe that, for some datasets, the utility of certain LLMs without defense is comparable to (or is even higher than) that under no attacks. This is because the Combined Attack is less effective for these LLMs. One possible reason is that these LLMs

Table 3: PISanitizer is effective against different injected tasks under GCG Attack. $\overline{\text{Utility}}$ represents the Utility for target tasks under no attacks and defenses.

Dataset	Injected Task	No Defense		PISanitizer		$\overline{\text{Utility}}$
		Utility	ASR	Utility	ASR	
Qasper	Target answer	0.12	0.96	0.28	0.02	0.32
	Hacked	0.04	0.61	0.30	0.0	
	Classic NLP task	0.03	0.87	0.29	0.01	
	General inst.	0.03	0.78	0.30	0.0	
HotpotQA	Target answer	0.12	0.96	0.56	0.02	0.59
	Hacked	0.15	0.30	0.58	0.0	
	Classic NLP task	0.03	0.86	0.59	0.0	
	General inst.	0.03	0.70	0.59	0.0	
GovReport	Target answer	0.02	1.0	0.34	0.0	0.34
	Hacked	0.18	0.19	0.34	0.0	
	Classic NLP task	0.02	0.87	0.34	0.0	
	General inst.	0.23	0.76	0.34	0.0	
MultiNews	Target answer	0.03	1.0	0.28	0.0	0.28
	Hacked	0.19	0.20	0.28	0.0	
	Classic NLP task	0.10	0.51	0.28	0.0	
	General inst.	0.18	0.67	0.28	0.0	
LCC	Target answer	0.10	0.82	0.38	0.0	0.42
	Hacked	0.30	0.21	0.39	0.0	
	Classic NLP task	0.24	0.39	0.36	0.0	
	General inst.	0.12	0.67	0.38	0.0	
Passage Retrieval	Target answer	0.05	0.94	0.97	0.0	1.0
	Hacked	0.31	0.34	0.97	0.0	
	Classic NLP task	0.09	0.82	0.96	0.0	
	General inst.	0.29	0.57	0.97	0.0	

Table 4: Compare PISanitizer with baselines. The best results are bold. The runtime is averaged over 6 datasets. The unit for runtime is second.

Attack	Defense	Qasper		HotpotQA		GovReport		MultiNews		LCC		Pass.Retri.		Runtime
		Utility	ASR	Utility	ASR	Utility	ASR	Utility	ASR	Utility	ASR	Utility	ASR	
Without Attack	No Defense	0.32	0.0	0.59	0.0	0.34	0.0	0.28	0.0	0.42	0.0	1.0	0.0	9.55
	Sandwich Prev.	0.37	0.0	0.62	0.0	0.34	0.0	0.27	0.0	0.34	0.0	0.99	0.01	12.37
	Instru. Prev.	0.33	0.0	0.61	0.0	0.34	0.0	0.28	0.0	0.41	0.0	0.98	0.0	9.64
	Meta-SecAlign	0.23	0.0	0.57	0.0	0.33	0.0	0.27	0.0	0.24	0.0	0.99	0.0	14.85
	PromptArmor	0.32	0.0	0.59	0.0	0.34	0.0	0.28	0.0	0.42	0.0	1.0	0.0	10.75
	PromptLocate	0.09	0.0	0.46	0.0	0.29	0.0	0.23	0.0	0.24	0.0	0.27	0.02	761.20
	DataFilter	0.20	0.01	0.48	0.03	0.31	0.0	0.27	0.0	0.27	0.0	0.33	0.05	15.86
	PISanitizer	0.32	0.0	0.59	0.0	0.34	0.0	0.28	0.0	0.42	0.0	1.0	0.0	10.14
Combined Attack	No Defense	0.15	0.92	0.24	0.66	0.03	0.97	0.05	0.91	0.15	0.73	0.67	0.27	10.05
	Sandwich Prev.	0.25	0.77	0.42	0.46	0.03	0.98	0.03	0.98	0.18	0.68	0.51	0.42	13.65
	Instru. Prev.	0.15	0.97	0.19	0.71	0.12	0.70	0.03	1.0	0.16	0.71	0.63	0.28	10.59
	Meta-SecAlign	0.17	0.58	0.26	0.56	0.32	0.0	0.27	0.0	0.23	0.14	0.88	0.12	15.01
	PromptArmor	0.17	0.74	0.30	0.53	0.21	0.44	0.09	0.75	0.18	0.66	0.70	0.24	13.78
	PromptLocate	0.09	0.0	0.43	0.05	0.29	0.0	0.23	0.0	0.24	0.0	0.31	0.03	633.61
	DataFilter	0.20	0.28	0.36	0.27	0.23	0.30	0.21	0.24	0.28	0.0	0.26	0.18	16.77
	PISanitizer	0.31	0.0	0.59	0.01	0.34	0.0	0.28	0.0	0.37	0.0	0.98	0.01	12.03
GCG Attack	No Defense	0.12	0.96	0.12	0.96	0.02	1.0	0.03	1.0	0.10	0.82	0.05	0.94	11.13
	Sandwich Prev.	0.16	0.93	0.10	1.0	0.02	1.0	0.03	1.0	0.05	0.92	0.0	1.0	13.90
	Instru. Prev.	0.08	1.0	0.10	1.0	0.02	1.0	0.03	1.0	0.05	0.92	0.0	1.0	11.15
	Meta-SecAlign	0.12	1.0	0.10	1.0	0.04	1.0	0.06	0.91	0.09	0.85	0.0	1.0	16.05
	PromptArmor	0.11	0.95	0.16	0.82	0.02	1.0	0.03	0.99	0.05	0.93	0.13	0.80	14.25
	PromptLocate	0.09	0.02	0.36	0.05	0.29	0.0	0.23	0.0	0.23	0.07	0.23	0.05	748.08
	DataFilter	0.17	0.19	0.31	0.28	0.23	0.22	0.22	0.21	0.24	0.13	0.27	0.14	18.51
	PISanitizer	0.28	0.02	0.56	0.02	0.34	0.0	0.28	0.0	0.38	0.0	0.97	0.0	13.92

exhibit robustness to prompt injection, e.g., GPT-4o-mini has been trained by OpenAI to resist prompt injection [62, 16].

PISanitizer is effective against different types of injected tasks: Table 3 demonstrates PISanitizer is consistently effective across various injected task types, including simple yet strong direct-output attacks (target answer generation, hacked generation) and more complex general-purpose instruction-following attacks (classic NLP task, general instruction).

PISanitizer outperforms baselines: We compare PISanitizer with baselines under state-of-the-art heuristic attack (Combined Attack) and optimization-based attack (GCG Attack). The results in Table 4 show that PISanitizer outperforms baselines.

Sandwich Prevention and Instructional Prevention cannot effectively prevent prompt injection, as they are based on simple heuristics.

Meta-SecAlign also cannot effectively defend against prompt injection to long context. For instance, the ASR is very high under GCG attack. The reason is that Meta-SecAlign aims to prevent an LLM from following the injected instruction, which can be very challenging when an attacker can perform an optimization-based attack. By contrast, PISanitizer can reduce ASR to nearly zero. The reason is that PISanitizer takes a fundamentally different strategy: PISanitizer designs a sanitization instruction to intentionally let an LLM follow any instruction in a context and sanitize tokens that drive instruction following behavior. An injected instruction that an LLM is more likely to follow is also more likely to be sanitized.

PromptArmor is also less effective as it directly prompts an LLM to detect and remove injected tokens, which can be challenging for long contexts. PromptLocate can also reduce ASR to nearly zero, but at the cost of utility loss. For instance, the utility of PromptLocate degrades without attacks. The reason is that PromptLocate would remove many clean tokens for a context (falsely) detected as contaminated.

Table 5: Detection-based defenses are less effective in long context scenarios under Combined Attack.

Dataset	DataSentinel		PromptGuard		AttentionTracker	
	FPR	FNR	FPR	FNR	FPR	FNR
Qasper	1.0	0.0	0.0	0.96	1.0	0.0
HotpotQA	1.0	0.0	0.0	0.87	1.0	0.0
GovReport	1.0	0.0	0.0	0.93	1.0	0.0
MultiNews	1.0	0.0	0.0	0.50	1.0	0.0
LCC	1.0	0.0	0.0	0.84	1.0	0.0
Pass. Retri.	1.0	0.0	0.0	0.99	1.0	0.0

Table 6: Combining detection-based defense and context attribution is less effective. The dataset is HotpotQA.

Defense	Without Attack		Combined Attack	
	Utility	ASR	Utility	ASR
DataSentinel+AttnTrace	0.40	0.05	0.41	0.05
PromptGuard+AttnTrace	0.59	0.0	0.29	0.56
AttentionTracker+AttnTrace	0.40	0.05	0.41	0.05
PISanitizer	0.59	0.0	0.59	0.01

DataFilter trains a sequence-to-sequence model to filter out injected instructions in a context. DataFilter is mainly designed for agentic applications with relatively short contexts. Our results show that DataFilter is less effective under long context scenarios, potentially due to the limited generalization of the trained model. We note that, without attacks, the ASR of DataFilter can even slightly increase compared to no defense. This is because the ground truth answers for a few target questions are binary (e.g., “yes” or “no”). When a defense causes an LLM to generate an incorrect answer for these questions (e.g., due to randomness or critical information removal), the ASR becomes non-zero.

PISanitizer is efficient: Table 4 also reports the runtime of different defense methods. Overall, PISanitizer achieves comparable efficiency to other defenses, except for PromptLocate, which is significantly slower because it needs to classify many segments when the context is long. We further measure the computation time of PISanitizer to sanitize contexts. On average, it takes around 1.8s for PISanitizer to sanitize a very long input containing thousands of tokens. This demonstrates that PISanitizer is computationally efficient and practical for real-world use.

Existing detection-based defenses are less effective: As shown in Table 5, state-of-the-art detection-based defenses cannot effectively detect prompt injection under long context, demonstrating that these defenses do not transfer effectively to long-context scenarios.

As state-of-the-art detection-based defenses [29, 31, 32] are less effective, existing attribution-based defenses [34, 35] are also insufficient when extended to sanitize injected tokens after detection. For instance, DataSentinel has a high FPR. As shown in Table 6, when combined with DataSentinel, the attribution-based defense AttnTrace [35] would remove many clean tokens from the context, thereby leading to utility loss. Similarly, due to the high FNR of PromptGuard, AttnTrace cannot effectively defend against prompt injection when combined with PromptGuard.

5.3 Ablation Study

We perform an ablation study for PISanitizer under our default setting (using HotpotQA for target tasks; using target answer generation as injected tasks).